

Projet Final — Développer pour le Cloud · Master 2

GreenLogistics

Sujet A — Tracking temps réel de livraison dernière mile

kind (3 nœuds)

ArgoCD · GitOps

Redpanda

Vault + ESO

Linkerd (mTLS)

Prometheus / Grafana

Kubecost

Soutenance — 6 juillet 2026 · Équipe GreenLogistics · Stack 100 % locale

01 Contexte métier

Startup de **livraison écologique du dernier kilomètre** : suivre les colis en temps réel et prévenir le destinataire à l'approche du livreur.

Destinataire

Notification « livraison imminente »
fiable — < 5 min avant l'arrivée.

Exploitant

Qualité de service : taux d'erreur API
< 1 %, latence P95 < 200 ms.

Plateforme

Déployer sans risque (GitOps),
observer, maîtriser les coûts.

02 Architecture — 4 microservices



Communication **synchrone** (HTTP via Ingress) + **asynchrone** (Redpanda, compatible Kafka) → le pic GPS n'impacte pas l'API colis.

03 Les quatre services

Service	Runtime	Rôle	IO
parcel-api	Python 3.12 / FastAPI	CRUD colis, statuts, événements	PostgreSQL (asyncpg)
gps-ingestor	Python 3.12 / FastAPI	Ingestion points GPS	Producteur Redpanda
notifier	Node.js 20 / Express	Distance haversine → notifie	Consumer Redpanda → MailHog
tracker-front	React (Vite) → NGINX	SPA publique de suivi	Appelle parcel-api

🔍 **Honnêteté d'ingénierie** : `parcel-api` et `gps-ingestor` ont été migrés de Node → Python/FastAPI en cours de projet (async + `prometheus_client`). Les 34 artefacts Node morts ont été purgés.

04 Conteneurisation — Docker

Chaque service est **emballé** avec son langage et ses librairies : il tourne à l'identique en local, en CI et dans le cluster — fin du « ça marche sur ma machine ».

- ▶ **Image** = le moule figé · **conteneur** = le moule en exécution
- ▶ **Dockerfile** = la recette (1 ligne = 1 couche)
- ▶ **Multi-stage** : atelier de build → image finale légère

Durcissement des images

- ✓ **Non-root** (USER appuser)
- ✓ HEALTHCHECK intégré
- ✓ Base slim/alpine — surface d'attaque réduite

05 Orchestration — Kubernetes

Modèle déclaratif : on décrit l'état voulu, Kubernetes le maintient en permanence (**self-healing**) sur le cluster kind (3 nœuds).

Deployment

garde N répliques vivantes de l'image

Service

adresse DNS fixe + load-balancer
devant les Pods

Probes

readiness = prêt au trafic · *liveness* =
à redémarrer

🔗 Objets reliés par **labels** (app: `parcel-api`), jamais par des flèches. Les `requests/limits` CPU/RAM servent de base au **HPA**.

06 Plateforme en code — Terraform (IaC)

Toute la plateforme (ArgoCD, Prometheus, Vault, Redpanda...) est **provisionnée par code**, reproductible en une commande. Cycle `init` → `plan` → `apply`.

- ▶ `helm_release` = installe un chart (ArgoCD, kube-prometheus-stack...)
- ▶ `module namespace-rbac` = namespaces + ServiceAccount + RBAC
- ▶ `.tfstate` = la mémoire de l'infra (backend local)

Terraform vs ArgoCD

Terraform pose les fondations — et installe ArgoCD lui-même.

ArgoCD déploie ensuite l'application en continu (GitOps).

07 Décisions d'architecture (ADR)

ADR	Décision	Raison clé
001	Redpanda comme broker	API Kafka, sans ZooKeeper, plus léger
002	PostgreSQL StatefulSet	Données structurées, UUID natif
003	Linkerd plutôt qu'Istio	mTLS auto, ~400 Mo vs ~1 Go — critique en local
004	CI/CD pull-based (ArgoCD)	Aucun kubeconfig exposé dans la CI
005	Vault + External Secrets	Zéro secret en clair dans Git

08 CI/CD sécurisé — pull-based

Le principe : à chaque push, un robot teste, construit et scanne les images — puis met à jour le déploiement. La CI ne touche jamais au cluster.

GitHub Actions (matrice ×4)

- ✓ Lint + tests (ruff/pytest · eslint/vitest)
- ✓ Build multi-arch → GHCR
- ✓ **Trivy** : échec si CVE **CRITICAL** corrigeable
- ✓ `update-manifests` réécrit le tag SHA

Déploiement GitOps

- ✓ La CI **ne touche jamais** au cluster
- ✓ **ArgoCD** tire le nouveau tag et déploie
- ✓ Secrets via **Vault + ESO**, hors Git

```
push → CI (test/build/scan) → commit tag SHA → ArgoCD sync → cluster
```

09 GitOps — App of Apps

Git est la **seule source de vérité** : ArgoCD compare Git ↔ cluster en boucle et corrige (**selfHeal**, **prune**). Zéro `kubectl apply` manuel.

La racine `greenlogistics` déploie 7 Applications enfants :

- ▶ `infra` (Vault, ESO, netpol...)
- ▶ `parcel-api` · `gps-ingestor`
- ▶ `notifier` · `tracker-front`
- ▶ `monitoring` (SLO, dashboard, alerte)
- ▶ `seed` (données de démo)

Déploiement progressif

Self-Heal + **prune** activés → démo live : scale à 0 → resync auto (~90 s).

Canary : `parcel-api` en Argo Rollouts, montée en charge progressive + rollback auto sur métriques.

10 Observabilité & SRE

Les services exposent `/metrics` → Prometheus collecte → Grafana affiche → Alertmanager alerte (MailHog). Objectifs de qualité mesurés par des **SLO**.

- ✓ kube-prometheus-stack (Prom + Grafana + Alertmanager)
- ✓ Loki + Promtail — logs centralisés
- ✓ `/metrics` scrapé via **ServiceMonitor**
- ✓ **Dashboard Grafana custom SLO**
- ✓ **Alerte → MailHog** (vérifiée bout-en-bout)

2 SLO en Recording Rules

- ▶ Taux d'erreur `parcel-api` < 1 %
- ▶ Latence P95 < 200 ms
- ▶ + Error Budget restant

0 %

Taux d'erreur

~5 ms

Latence P95

100 %

Error Budget

11 Sécurité (DevSecOps) & FinOps

Sécurité

- ✓ Trivy bloquant (CVE CRITICAL)
- ✓ **NetworkPolicies** deny-by-default + allow-lists
- ✓ **mTLS** automatique via Linkerd
- ✓ Images non-root, HEALTHCHECK, multi-stage
- ✓ RBAC : SA + Role/RoleBinding (module TF)

FinOps

- ✓ **Kubecost** — coût par namespace
- ✓ Labels team/env/app
- ✓ requests/limits sur tous les pods
- ✓ **HPA** parcel-api 2→6 · gps-ingestor 2→10

▶ Démo live

- ▶ 1 Chaîne métier : colis → OUT_FOR_DELIVERY → GPS → email MailHog
- ▶ 2 ArgoCD Self-Heal : scale à 0 → resync auto
- ▶ 3 HPA sous charge (trafic → montée des replicas)
- ▶ 4 Grafana : dashboard SLO en temps réel
- ▶ 5 Alerte SLO → email [FIRING] dans MailHog
- ▶ 6 Kubecost : coût par namespace

12 Retour d'expérience — incidents résolus

Prometheus OOMKilled

196 restarts, tout le monitoring mort. Limite 512Mi dépassée au chargement du WAL (253 segments). → limite portée à **1 Gi**. Stable, 0 restart.

Métriques absentes

Aucun ServiceMonitor + NetworkPolicy bloquant le scrape. → 2 SM + netpol allow-metrics-scrape.

SLO PromQL faux

`rate(erreurs) vide sans 5xx · quantile non agrégé. → or on() vector(0) + sum by (le).`

ExternalSecret Degraded

Vault dev-mode réinitialisé au restart → secret re-provisionné. Procédure documentée.

13 Résultats — état du cluster

- ✓ **ArgoCD** : 7 applications **Synced / Healthy**
- ✓ **SLO** parcel-api : erreur 0 %, P95 $\approx$ 5 ms, Error Budget 100 %
- ✓ **HPA** actifs (parcel-api 2/6, gps-ingestor 2/10)
- ✓ **Prometheus** stable, cibles parcel-api / gps-ingestor **up**
- ✓ **Alerte** livrée dans MailHog — [FIRING:1] ... (critical greenlogistics)
- ✓ **Reproducible** sur machine tierce en < 30 min (SETUP.md)

14 Pistes d'amélioration

- ▶ Tags **sémantiques** d'images sur release (vX.Y.Z) en plus du SHA
- ▶ Endpoint `/metrics` sur `notifier` → SLO de consommation d'événements
- ▶ SLO **multi-fenêtres** (burn-rate alerting) plutôt qu'un seuil simple
- ▶ Vault en mode **persistant** (non-dev) → plus de re-provisionnement
- ▶ Bonus : Chaos Mesh · tracing Tempo/OpenTelemetry · e2e (k6/Playwright) en CI

Merci

Questions ?

7 apps Synced/Healthy

SLO tenus

Alerte vérifiée

100 % local & reproductible

Dépôt : github.com/boubaLaria/dev-cloud-tp-final · tag v1.0–soutenance